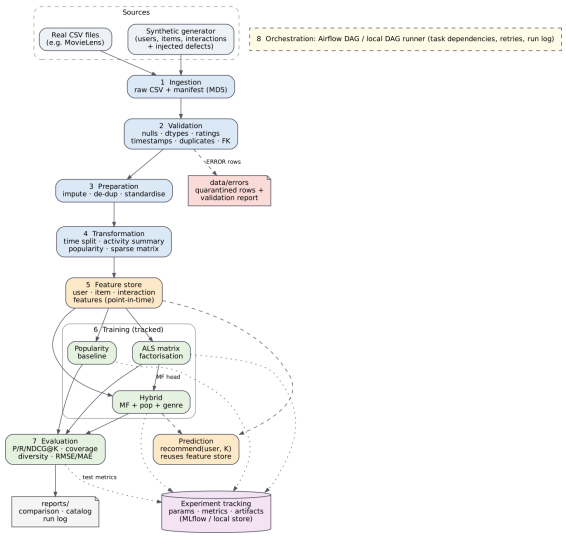
End-to-End Data Management Pipeline for a Recommendation System

DM4ML Assignment 2 | Final report | Stages: ingestion → validation → preparation → transformation → feature management → training → evaluation → orchestration

1. Problem, data and design decisions

The pipeline recommends items to users of a movie-style platform from user, item and interaction (rating + timestamp) tables. **Data.** The build environment had no internet access, so a **documented synthetic generator** (1200 users, 600 items, 48,029 timestamped ratings after cleaning) is used. It has real structure (user taste vectors, genre preferences, item quality, long-tailed popularity, choice of items depends on taste) so personalisation is learnable, and it **deliberately injects defects** (about 1.6% of interaction rows plus defects in users/items) with a ground-truth log so validation can be checked. A csv mode accepts real data such as MovieLens mapped to the same three tables.

Key decisions. (1) *Contract-first stages:* each stage reads/writes files in a fixed folder (raw → interim → processed → features), so any stage can be re-run alone and outputs are inspectable. (2) *Severity-based validation:* ERROR rows are quarantined, WARN fields are blanked and imputed, never silently dropped. (3) *Time-based split* (newest 20% of interactions = test) instead of random, to avoid look-ahead leakage; features are computed *point-in-time* from training data only. (4) *One feature layer* shared by training, evaluation and prediction, with a registry that is asserted against the code. (5) *Tuning only on a validation window* carved from the end of the training period; the test set is touched once, for reporting.



Concern	Tool used	Why
Data / compute	Pandas, NumPy, SciPy sparse	Small data; sparse matrix for CF
Validation	Custom rule engine (src/validation)	Great Expectations/Pandera not installable offline; rules map 1:1 to expectations
Features	Custom file-backed store + YAML registry	Same API shape as Feast (build / get_features); no DB needed
Models	Own ALS-MF, popularity, hybrid (NumPy)	No Surprise/Implicit offline; transparent and unit-testable
Tracking	MLflow if installed, else MLflow-style file store	Same params/metrics/artifacts either way
Orchestr.	DAG runner + Airflow DAG file	Runner executed here; Airflow file shares the task graph
Storage	CSV (+ .npz sparse matrix)	Parquet is a drop-in swap when pyarrow is available

Figure 1 – Pipeline architecture (left) and tool choices (right).

2. Validation, preparation and features

Validation covers missing values, duplicates, data types, rating range [1–5], timestamp validity (unparsable / future / before signup) and referential integrity. Of 51,522 input rows, **1,709 were quarantined** to data/errors/ with the violated rule(s) (users 23, items 8, interactions 1678); row counts reconcile exactly (*in = valid + quarantined* is asserted). Every injected defect type was detected, with small differences explained in the report (see reports/data_validation_report.md); orphan counts exceed the injected number because rejecting a user/item row cascades to its interactions - a real referential-integrity effect. Preparation then imputed 45 ages (median), 40 countries, 20 titles and 10 release years, and standardised gender/country/genre formats (60 genre strings fixed).

Transformation. Time cut-off 2024-10-24: 38,423 train / 9,606 test rows; derived datasets are the user activity summary, item popularity table and a 1187×597 sparse user-item matrix (density 5.4%). **Features** (23 catalogued in reports/feature_catalog.md): user (activity count, average rating, rating spread, like-ratio, days since last interaction, tenure, preferred category and its share), item (popularity, rank, average and Bayesian-shrunk rating, like-ratio, age, recency) and user×item interaction features (genre affinity, rating-vs-user-bar gap, preferred-category match). Cold-start users/items receive documented defaults, so prediction never fails on unseen IDs.

3. Model results (test period, K = 10; K = 5 in the CSV/MD)

Model	P@10	R@10	NDCG@10	Cov@10	Div@10	RMSE	MAE
Popularity (baseline)	0.0374	0.0883	0.0693	0.0720	0.8513 *	1.0558	0.8685
ALS matrix factorisation	0.0283	0.0697	0.0506	0.0519	0.7907	0.9624 *	0.7848 *
Hybrid (MF + pop + genre)	0.0623 *	0.1494 *	0.1170 *	0.1993 *	0.7139	0.9624 *	0.7848 *

* best in column. Evaluated on 1029 users with ≥1 liked (rating ≥ 4) test item; seen items excluded; RMSE/MAE on 9,606 test ratings (popularity uses a Bayesian item mean; the hybrid reuses MF's rating head).

Findings. The **hybrid improves on the popularity baseline** by 69% in NDCG@10 (0.069 → 0.117), 66% in Precision@10 and 69% in Recall@10, and recommends from 20% of the catalogue vs 7%. The price is lower genre diversity (0.71 vs 0.85), because it concentrates a user's list on their favourite genres. **MF alone** is the best *rating* predictor (RMSE 0.962 vs 1.056, 8.8% lower) but **ranks worse than popularity** (NDCG@10 0.051): ranking by predicted rating favours obscure items users rarely choose, while the test relevance depends on what users *select* as well as what they like. The hybrid fixes this by blending three per-user z-scored signals (weights from a grid search on a validation window: MF 0.5, popularity 1.5, genre affinity 0.5). Honest ablation: on the validation window, the same blend *without* MF reaches NDCG 0.0918 vs 0.0978 with it, so most of the gain comes from popularity + the feature-store genre affinity and MF adds a modest 7%. Absolute precision is low because each user has only ~4.7 liked items in the test window against a 597-item catalogue (a perfect list of 10 would still score P@10 ≈ 0.47); the comparison between models, not the absolute value, is the point.

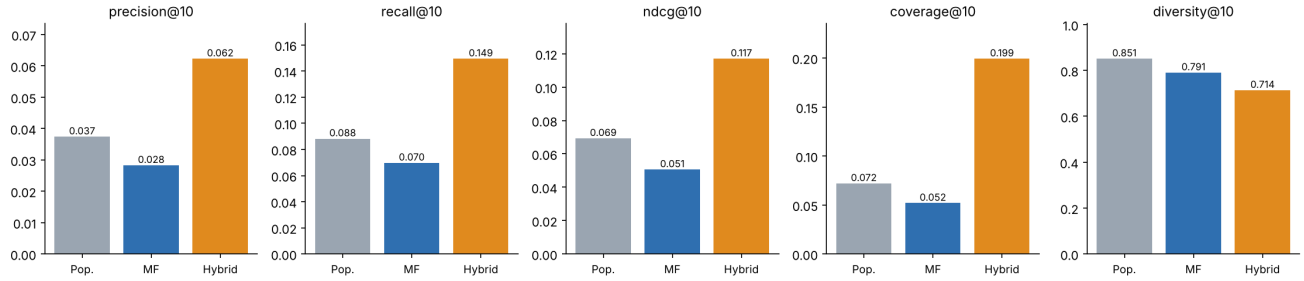


Figure 2 – Test-set comparison at the primary K.

4. Orchestration, reproducibility and tracking

The workflow is a 9-task DAG (src/orchestration/pipeline.py) with explicit dependencies (baseline and MF train in parallel branches; hybrid depends on MF; evaluation depends on all models). The runner executes in topological order, retries, skips downstream tasks on failure and writes pipeline_run_log.json. The same graph is generated into an Airflow DAG (dags/recommendation_pipeline_dag.py). Last run: **9/9 tasks succeeded in 11s**. Reproducibility: fixed seed, config file, fixed validation reference date, deterministic tie-breaking; two full runs from an empty state produced identical metrics. Each model run logs parameters (factors, regularisation, blend weights, seed, as-of time), training and test metrics, and the pickled model to the tracker; 7 unit tests cover validation rules, split leakage, point-in-time features, metrics (hand-computed) and the DAG order.

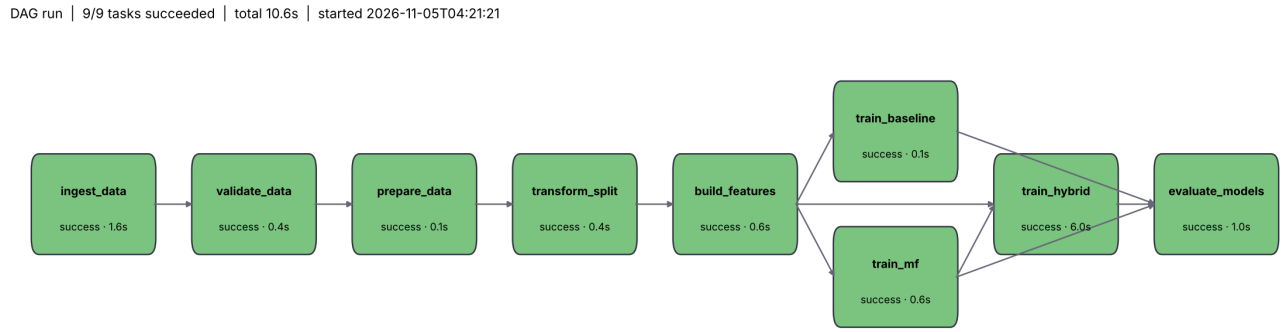


Figure 3 – Successful DAG run, rendered from reports/pipeline_run_log.json (task status and duration).

5. Limitations and next steps

(1) **Synthetic data**: absolute scores do not transfer to real platforms; the pipeline accepts MovieLens-style CSVs for that. (2) Airflow, MLflow and Great Expectations could not be installed offline: the Airflow DAG file was syntax-checked and its graph tested but not run in a scheduler, and the MLflow code path is untested here (the local file store with the same layout was used). The DAG screenshot is drawn from the real run log, not the Airflow UI. (3) Binary relevance and offline metrics ignore position bias and exposure; online A/B testing would be needed. (4) Items with no training history cannot be recommended on merit (cold-item problem); a content-based component using genres only partly covers this. (5) The ALS model is a plain explicit-feedback factoriser; implicit-feedback ALS or LightFM would likely rank better. (6) Features are materialised in batch, with no streaming/online store or drift monitoring.